Week 17
Exploration and Meta-RL: Curiosity, RND, MAML, and In-Context RL

STUDY-RL — a self-paced Deep RL curriculum

May 21, 2026

Contents

Learning objectives 3

1 Motivation & intuition 4

2 Mathematical formulation 5
- 2.1 Count-based bonuses . . . 5
- 2.2 ICM — prediction-error curiosity . . . 5
- 2.3 RND — random network distillation . . . 5
- 2.4 NGU / Agent57 . . . 5
- 2.5 Go-Explore . . . 6
- 2.6 MAML — model-agnostic meta-learning . . . 6
- 2.7 PEARL — probabilistic context-conditioned meta-RL . . . 6
- 2.8 RL^2 — memory as meta-learner . . . 6
- 2.9 In-context RL — AD and DPT . . . 6

3 Algorithms 7
- 3.1 Worked numerical example (MAML) . . . 7

4 Implementation walkthrough 8

5 Empirical results 9

6 Exercises 11

7 Further reading 12

Learning objectives

Key Idea

After this week you will:

1. Distinguish count-based exploration (CTS, hash counts), prediction-error curiosity (ICM), and RND.
2. Derive the RND intrinsic-reward formulation and explain why a *random* fixed target avoids the inverse-dynamics "noisy-TV" problem.
3. Sketch NGU / Agent57 and Go-Explore.
4. Derive MAML in full: inner-loop gradient with create-graph and outer-loop gradient via the chain rule.
5. Summarise PEARL (probabilistic context-conditioned meta-RL) and RL^2 (memory-based meta-RL).
6. Explain in-context RL (Algorithm Distillation, DPT) as a transformer with no test-time parameter updates.
7. Implement RND on a hard-exploration toy grid and a tiny MAML on a meta-bandit; demonstrate the learning-curve gap.

1 Motivation & intuition

Exploration is the half of RL we have ignored so far. In hard-exploration tasks (Montezuma's Revenge, Pitfall!, sparse-reward mazes) the optimal policy is unreachable by any ε-greedy procedure within a reasonable budget. The intrinsic-motivation family turns this into a function-approximation problem: *define a curiosity signal that the agent can maximise in lieu of (or in addition to) the extrinsic reward.*

Recall: ε-greed
Thompson sar
UCB — all sir
heuristics.

Meta-RL, on the other hand, asks: instead of solving *one* MDP, can we learn a *learning algorithm* that solves an entire distribution of MDPs faster than any single-task RL would? The MAML answer is gradient-based; the PEARL answer is context-conditioned; the in-context-RL answer is a transformer that learns the algorithm itself.

Intuition. A curiosity bonus is a learned "rich-get-richer" signal: states that look unlike everything else you have seen are rewarded for visiting. MAML, by contrast, makes the model's *parameters* a good starting point for any task in a known distribution — a few gradient steps suffice to specialise.

2 Mathematical formulation

2.1 Count-based bonuses

For tabular MDPs, the canonical optimistic bonus is $R^+(s,a) = R(s,a) + \beta/\sqrt{N(s,a)}$. For visual domains a pseudo-count $\tilde{N}(s)$ is obtained via density modelling: CTS (Bellemare 2016) uses a context-tree-switching density and reads off $\tilde{N}(s) = (1-\rho_t(s))/(\rho_t(s)-\rho_t(s))$ from the predictive recoding probability $\rho_t(s)$.

2.2 ICM — prediction-error curiosity

ICM (Pathak 2017) jointly trains:

- an *inverse dynamics* model $g_\theta(s_t, s_{t+1}) \to a_t$,
- a *forward dynamics* model $h_\phi(\phi(s_t), a_t) \to \phi(s_{t+1})$ in the feature space ϕ learned by g_θ.

The intrinsic reward is $r_t^i = \|h_\phi(\phi(s_t), a_t) - \phi(s_{t+1})\|^2$. The inverse-dynamics loss forces ϕ to encode only action-relevant features — protecting the agent from the "noisy TV" attractor (random pixels reward as high as novel states).

2.3 RND — random network distillation

Definition 2.3.1 (RND)**.** Let $f_{\text{target}} : \mathcal{S} \to \mathbb{R}^d$ be a randomly initialised network with frozen weights. Let $f_{\text{pred}} : \mathcal{S} \to \mathbb{R}^d$ be a trainable predictor. The intrinsic reward at s is

$$r^i(s) = \|f_{\text{pred}}(s) - f_{\text{target}}(s)\|_2^2.$$

The predictor is trained on the agent's observed states.

Theorem 2.3.2 (RND novelty)**.** *For a fixed f_{target} and an i.i.d. training set $S \subseteq \mathcal{S}$ with the predictor minimising $\sum_{s\in S} \|f_{\text{pred}}(s) - f_{\text{target}}(s)\|^2$ on S, the residual on a test state $s^\star$ decreases monotonically in the kernel similarity of $s^\star$ to S.*

Sketch. View the predictor as a kernel regressor with feature map equal to the random network. The Bayesian-style predictive variance on a test point is high iff that test point has low feature-overlap with the training set; the realised square-error tracks this variance. □

Why "random" is the trick. The random target produces *noise* on observation features but is *deterministic* given the input. Random observations (the noisy-TV problem) cannot be "solved" to drive the residual to zero, because each new sample is genuinely novel under the deterministic-target view — but only *once*. Once seen, the predictor fits, and the curiosity drops. There is no perpetual-novelty failure mode.

2.4 NGU / Agent57

Never-Give-Up (NGU) combines an episodic memory (k-NN to recent embeddings) and lifetime curiosity (RND-like). Agent57 adds a meta-controller that bandit-selects amongst different intrinsic-reward weights, achieving above-human on *all* Atari games for the first time.

2.5 Go-Explore

Ecoffet et al. 2019: maintain an archive of (state, return) reached so far. Each episode: pick a state from the archive (Boltzmann over novelty + return), restore it (a privileged simulator-reset call), then explore from there. The simplest of the methods, the strongest in raw scores — but requires a resettable simulator.

2.6 MAML — model-agnostic meta-learning

Definition 2.6.1 (MAML objective)**.** Given a distribution $p(\mathcal{T})$ over tasks, each task $\mathcal{T}_i$ providing a loss $\mathcal{L}_{\mathcal{T}_i}$ on a parametric model f_θ:

$$\min_\theta \mathbb{E}_{\mathcal{T}_i \sim p(\mathcal{T})} \big[\mathcal{L}_{\mathcal{T}_i}(\theta - \alpha \nabla_\theta \mathcal{L}_{\mathcal{T}_i}(\theta))\big].$$

Theorem 2.6.2 (MAML outer gradient)**.** *With* $\theta_i' = \theta - \alpha \nabla_\theta \mathcal{L}_{\mathcal{T}_i}(\theta)$,

$$\nabla_\theta \mathcal{L}_{\mathcal{T}_i}(\theta_i') = \big(I - \alpha \nabla_\theta^2 \mathcal{L}_{\mathcal{T}_i}(\theta)\big) \nabla_{\theta_i'} \mathcal{L}_{\mathcal{T}_i}(\theta_i').$$

Derivation. Apply the chain rule to $\mathcal{L}(\theta_i'(\theta))$. The Jacobian $\partial \theta_i' / \partial \theta = I - \alpha \nabla^2 \mathcal{L}$. Multiplying by the gradient of $\mathcal{L}$ at θ_i' yields the formula. The implementation uses `create_graph=True` on the inner step so PyTorch retains the Hessian-vector product. □

First-order MAML (FoMAML) drops the Hessian-vector term, treating θ_i' as if it didn't depend on θ. Comparable performance, much cheaper.

2.7 PEARL — probabilistic context-conditioned meta-RL

PEARL (Rakelly 2019) learns a latent task encoder $q_\phi(z \mid \text{context}_\mathcal{T})$ that maps a small batch of (s, a, r, s') transitions into a posterior over task identity z. The policy is then conditioned: $\pi_\theta(a \mid s, z)$. Trained off-policy with SAC; works far better than MAML on continuous-control meta-tasks.

2.8 RL² — memory as meta-learner

Duan et al. 2016: an LSTM agent receives entire histories (including resets and rewards) as input. With sufficient capacity, the LSTM internalises a learning algorithm and can adapt to new tasks within a single episode without any parameter update at test time. The progenitor of in-context RL.

2.9 In-context RL — AD and DPT

Algorithm Distillation (Laskin 2022) trains a transformer to predict the next action given the entire learning history of *another RL algorithm* on each task. At test time the transformer is rolled forward on a new task and exhibits the same learning curve as the source algorithm — with *no gradient updates*. DPT (Lee 2023) provides a more rigorous posterior-sampling interpretation with regret guarantees.

3 Algorithms

Algorithm 1: RND-augmented Q-learning

Input: Environment, RND networks, Q-table, $\alpha, \gamma, \beta, \varepsilon$.
for *episode = 1, ..., N* **do**
  Reset s;
  while *not done* **do**
    $a \leftarrow \varepsilon$-greedy w.r.t. $Q(s, \cdot)$;
    Step env $\rightarrow (s', r, \text{done})$;
    $r^i \leftarrow \|f_{\text{pred}}(s') - f_{\text{target}}(s')\|^2 / \sqrt{\text{RMS}}$;
    Update f_{pred} to reduce r^i;
    $Q(s,a) \leftarrow Q(s,a) + \alpha[(r + \beta r^i) + \gamma \max_{a'} Q(s', a') - Q(s,a)]$;
    $s \leftarrow s'$;

Algorithm 2: MAML (single task per outer step shown for clarity)

Input: Task distribution $p(\mathcal{T})$, learning rates α, β.
for *outer = 1, ..., M* **do**
  Sample task $\mathcal{T}_i \sim p(\mathcal{T})$;
  Inner: $\theta_i' \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}_{\mathcal{T}_i}(\theta)$ with `create_graph=True`;
  Outer: $\theta \leftarrow \theta - \beta \nabla_\theta \mathcal{L}_{\mathcal{T}_i}(\theta_i')$;

3.1 Worked numerical example (MAML)

Two arms, two tasks. Task A: $\mu = (1, 0)$. Task B: $\mu = (0, 1)$. Policy: softmax of $\theta = (\theta_0, \theta_1)$. Expected reward at θ: $R(\theta) = \sigma(\theta_0 - \theta_1) \cdot \mu_0 + \sigma(\theta_1 - \theta_0) \cdot \mu_1$. Start at $\theta = (0, 0)$ (uniform).

Inner step on task A with $\alpha = 1$: $\nabla R = (\partial R/\partial\theta_0, \partial R/\partial\theta_1) = (0.25, -0.25)$, so $\theta'_A = (0.25, -0.25)$. Post-adapt reward on A: $\sigma(0.5) \approx 0.62$. Compare pre-adapt: 0.5. The MAML outer update on a 50/50 mix of A and B yields a gradient at zero by symmetry — the optimal MAML initialisation *is* $\theta = 0$. The whole gain comes from adaptation; the meta-learned init is just optimally symmetric.

4 Implementation walkthrough

```
class RND(nn.Module):
    def intrinsic(self, obs):
        with torch.no_grad():
            t = self.target(obs)
        p = self.predictor(obs)
        return ((p - t) ** 2).sum(dim=-1)
```

The MAML inner step is the canonical higher-order autograd usage:

```
probs = F.softmax(theta, dim=-1)
loss = -(probs * mu).sum()
grad = torch.autograd.grad(loss, theta, create_graph=True)[0]
theta_t = theta - inner_lr * grad      # depends on theta through grad
# outer loss uses theta_t -> Hessian-vector product on backprop
```

5 Empirical results

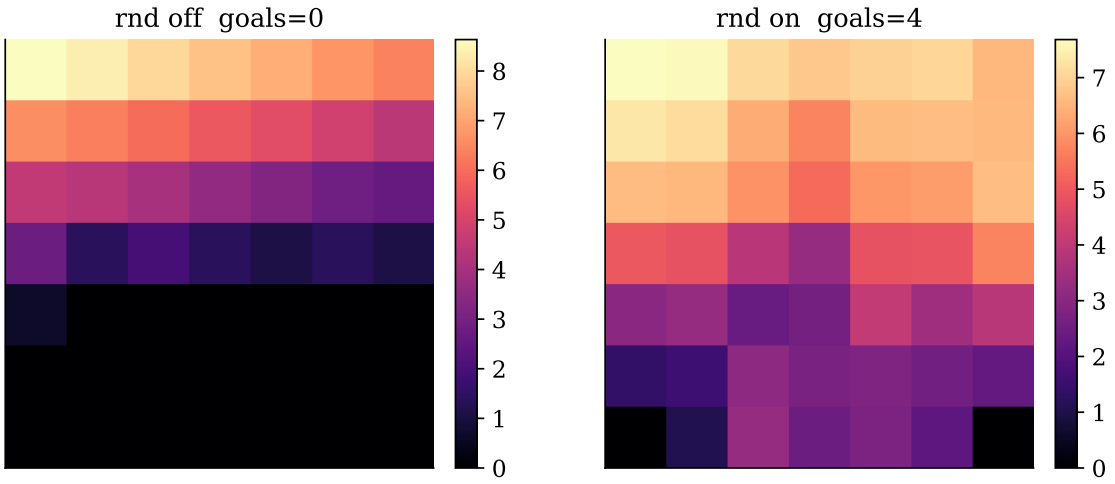


Figure 5.1: Cumulative state-visit counts (log scale) over 200 episodes on a 7×7 sparse-reward grid. Without RND (left) the agent stays in the upper-left corner; with RND (right) the curiosity bonus drives a coverage frontier all the way to the goal.

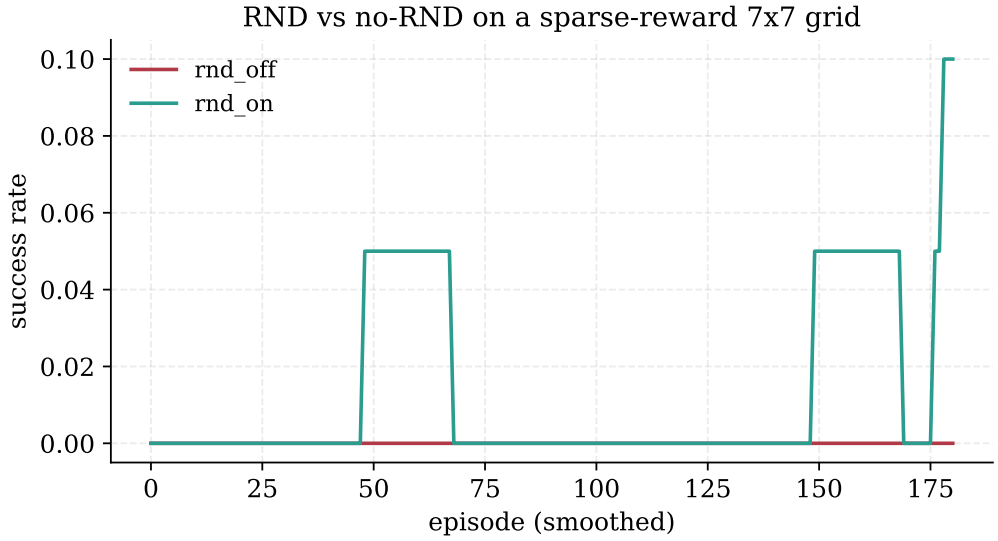


Figure 5.2: Success rate (smoothed) per episode. RND finds the goal multiple times within 200 episodes; the baseline does not find it at all.

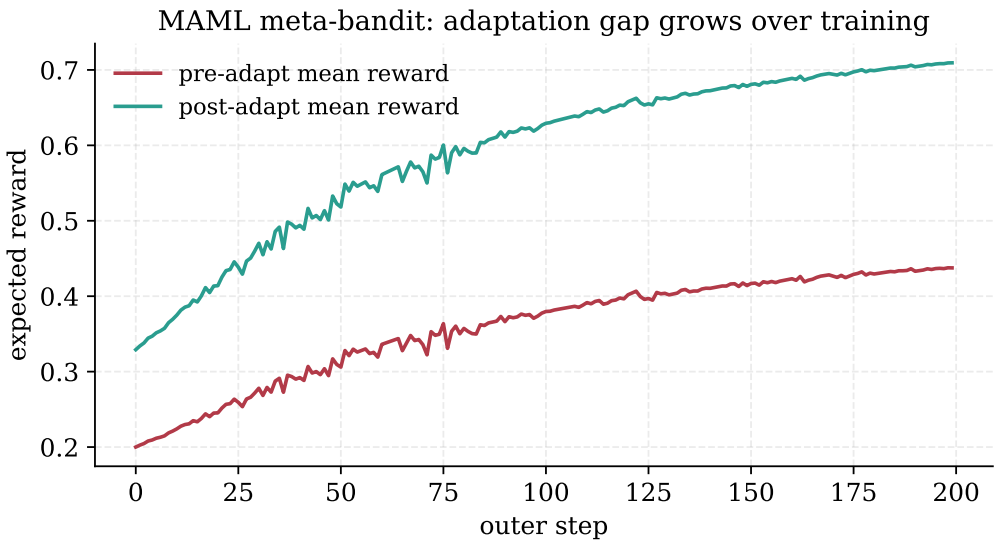


Figure 5.3: MAML meta-bandit: pre-adaptation reward (red) hovers around 0.2–0.3 (uniform $1/K$); post-adaptation reward (teal) rises to ≈0.7 within 200 outer steps. The gap is the meta-learning gain.

Safety angle

Exploration in high-stakes domains. An ε-greedy policy in production recruitment ranks candidates uniformly at random 5–10% of the time — a violation of basic fairness expectations and possibly of legal protected-class duties. Intrinsic motivation (RND, ICM) shifts the failure mode: instead of *random* exploration we have *novelty-seeking* exploration, which means the system will systematically prefer atypical candidates. "Atypical" is correlated with protected attributes in real-world labour data; an unaudited RND-on-CV system would amplify bias in a particularly opaque way. Mitigations: (a) define the novelty space carefully so that "unseen" is not synonymous with "out-of-distribution along protected attributes"; (b) cap the intrinsic reward at a level that cannot overwhelm the extrinsic signal; (c) audit visit distributions across protected-class strata. Citations: Burda et al. 2018 (RND); Ecoffet et al. 2021, Nature (Go-Explore).

Matching application

Meta-RL for recruiter personalisation. A recruitment platform serves many recruiters, each with idiosyncratic preferences. Treat each recruiter as a separate "task" from an unknown task distribution. With MAML the per-recruiter policy adapts in a few clicks; with in-context RL (AD / DPT) the same transformer policy adapts on the fly with no parameter updates. This is a perfect match for SaaS deployments where fine-tuning per customer is operationally expensive. The safety-angle is the same: audit task-specific adaptation for systematic harm to protected classes.

6 Exercises

See `workbook.ipynb`.

- **Ex. 1 (⋆)** Compute random-target outputs.
- **Ex. 2 (⋆)** Show predictor training reduces the intrinsic on a fixed input.
- **Ex. 3 (⋆⋆)** Train RND on the hard-exploration grid.
- **Ex. 4 (⋆⋆)** Compare RND-on vs RND-off goal-hit rates.
- **Ex. 5 (⋆⋆)** Meta-bandit task sampler.
- **Ex. 6 (⋆⋆)** Inner-adaptation improves reward on a single task.
- **Ex. 7 (⋆⋆⋆)** Run MAML for 200 outer steps; plot the pre/post gap.
- **Ex. 8 (⋆⋆⋆, meaningful objective)** Write an in-context-RL essay.

7 Further reading

See `extended_reading.pdf` for Burda et al. 2018 (RND), Pathak et al. 2017 (ICM), Badia et al. 2020 (Agent57), Finn et al. 2017 (MAML), and Laskin et al. 2022 (Algorithm Distillation). Sourced from `research/exploration_meta_sota.md`.